

Instituto Tecnológico de Costa Rica
Centro Académico de Alajuela
IC-4700 Lenguajes de Programación

II Semestre, 2026



Proyecto #1: Procesamiento Paralelo de Imágenes con Scheme y Erlang

Estudiantes:

José David Quirós González
Carné: 2025065801

Carlos Andrés Zúñiga Herrera
Carné: 2025065194

Profesor:

Eddy Miguel Ramírez Jiménez

Fecha de entrega:

23 de setiembre de 2026

Índice

1. Introducción	2
2. Arquitectura del sistema	2
3. Representación y lectura de imágenes	3
4. División de la imagen en regiones	3
4.1. El problema del halo	4
5. Manejo de los bordes de la imagen	4
6. Modelo de concurrencia	5
7. Protocolo de comunicación entre Erlang y Scheme	5
7.1. Detección de fallos	6
8. Implementación del filtro	6
8.1. Filtro gaussiano (obligatorio)	6
8.2. Inversión de colores (transformación adicional)	6
9. Limitaciones del sistema	7
10.Resultados experimentales	7
10.1. Filtro gaussiano	7
10.2. Filtro de inversión de colores	8
11.Análisis del speedup y la eficiencia	8
11.1. Comportamiento general	8
11.2. Gaussiano vs. inversión: el costo relativo de la sobrecarga	8
11.3. Un caso de rendimiento decreciente	9
11.4. Factores que explican el speedup por debajo del ideal	9

1 Introducción

El presente proyecto tiene como objetivo desarrollar un sistema de procesamiento paralelo de imágenes mediante la integración de dos lenguajes de programación funcional: Scheme y Erlang. La solución busca aprovechar las características particulares de cada lenguaje, asignando a Erlang las tareas relacionadas con la concurrencia y coordinación del procesamiento, mientras que Scheme se encarga de realizar las transformaciones correspondientes sobre las imágenes.

Para el procesamiento se utiliza el formato de imagen PPM, específicamente P3, permitiendo representar cada píxel mediante sus componentes RGB y trabajar con la información de la imagen sin depender de bibliotecas externas de procesamiento. El sistema divide la imagen en diferentes regiones, las cuales pueden ser procesadas de manera concurrente por procesos independientes de Erlang. Cada uno de estos procesos se comunica con una instancia de Scheme encargada de aplicar el filtro correspondiente sobre la región asignada.

Como transformación principal se implementa un filtro de difuminado gaussiano (*Gaussian Blur*) mediante un kernel de convolución. Debido a que este tipo de filtro necesita considerar los píxeles vecinos, el sistema debe contemplar el manejo de los límites entre regiones y de los bordes de la imagen.

Además de obtener una imagen correctamente procesada, el proyecto permite estudiar la integración entre lenguajes, el paso de mensajes entre procesos y el impacto de la concurrencia sobre el rendimiento. Para ello, se comparan ejecuciones utilizando diferentes cantidades de procesos, permitiendo posteriormente analizar métricas como el tiempo de ejecución, el *speedup* y la eficiencia.

2 Arquitectura del sistema

La arquitectura del sistema se basa en la distribución de responsabilidades entre Erlang y Scheme. Erlang funciona como el encargado de coordinar la ejecución general del programa, mientras que Scheme se encarga del procesamiento de las regiones de la imagen mediante los filtros implementados.

El proceso inicia cuando Erlang recibe los parámetros necesarios para la ejecución y realiza la lectura de la imagen en formato PPM P3. A partir de la información obtenida, la imagen se divide en diferentes regiones de acuerdo con la cantidad de procesos indicada por el usuario.

Erlang crea procesos independientes para trabajar con las regiones generadas. Cada proceso recibe la información correspondiente a una región de la imagen y ejecuta una instancia independiente del programa desarrollado en Scheme. De esta manera, las diferentes regiones pueden ser procesadas de forma concurrente.

Scheme recibe la información necesaria para procesar la región asignada y aplica la transformación correspondiente. Para el filtro gaussiano, se utilizan los píxeles de la región junto con la información adicional necesaria de los píxeles vecinos. Una vez terminado el procesamiento, Scheme devuelve el resultado a Erlang.

Finalmente, Erlang recibe los resultados generados por los diferentes procesos y los utiliza para reconstruir la imagen procesada. Posteriormente, el sistema genera el archivo PPM correspondiente a la imagen de salida.

Esta separación permite que cada lenguaje se utilice de acuerdo con las características

requeridas por el proyecto: Scheme realiza las operaciones funcionales asociadas con la transformación de los píxeles, mientras que Erlang administra la creación de procesos, la distribución del trabajo, la comunicación y la coordinación de la ejecución concurrente.

3 Representación y lectura de imágenes

Para el manejo de las imágenes se utiliza el formato PPM (*Portable Pixmap Format*), específicamente su variante P3. Este formato permite representar una imagen mediante texto, facilitando la lectura y manipulación de sus valores sin necesidad de utilizar bibliotecas externas para el procesamiento de imágenes.

Un archivo PPM P3 contiene inicialmente un encabezado con la información básica de la imagen. El primer valor corresponde al identificador **P3**, seguido por el ancho y alto de la imagen y, posteriormente, el valor máximo que puede tomar cada componente de color, normalmente 255. Después del encabezado se encuentran los valores correspondientes a los píxeles de la imagen.

Cada píxel se representa utilizando tres componentes: rojo, verde y azul (RGB). Por lo tanto, un píxel puede representarse mediante una tupla de la siguiente forma:

$\{R, G, B\}$

donde cada componente contiene un valor entre 0 y 255. Por ejemplo, el píxel 255, 0, 0 representa el color rojo.

La lectura inicial del archivo es realizada por Erlang. El contenido del archivo PPM se separa en valores individuales para poder identificar el encabezado y posteriormente los componentes RGB. Una vez obtenidos los valores correspondientes a los colores, estos se agrupan de tres en tres para formar los píxeles de la imagen.

Posteriormente, los píxeles son organizados de acuerdo con las dimensiones obtenidas del encabezado del archivo. Esta representación facilita la división de la imagen en regiones y permite que cada proceso Erlang trabaje con una sección determinada antes de enviarla a Scheme para aplicar el filtro correspondiente.

Una vez finalizado el procesamiento de todas las regiones, los resultados pueden volver a organizarse respetando las dimensiones originales de la imagen para generar el archivo PPM de salida.

4 División de la imagen en regiones

Para permitir el procesamiento concurrente, la imagen se divide en regiones horizontales: cada región corresponde a un conjunto contiguo de filas completas. Esta estrategia se eligió por dos razones. Primero, dado que el formato PPM almacena los píxeles en orden *row-major* (fila por fila), cortar la imagen en bandas horizontales resulta directo una vez que los píxeles están organizados como una lista de filas, sin necesidad de manipular columnas individuales. Segundo, al dividir únicamente por filas, cada fila conserva siempre su ancho completo, lo cual simplifica el tratamiento de los bordes laterales de la imagen, como se explica más adelante.

La cantidad de filas asignadas a cada región se calcula mediante división entera entre la altura de la imagen y la cantidad de procesos solicitada. Cuando esta división no es exacta, el resto de filas se asigna íntegramente a la última región, de modo que ninguna

fila de la imagen queda sin procesar ni se procesa más de una vez. Adicionalmente, el sistema garantiza que nunca se creen más regiones que filas tiene la imagen: si se solicitan más procesos que filas disponibles, la cantidad efectiva de regiones se limita a la altura de la imagen, evitando así que múltiples procesos terminen operando sobre una región vacía o indefinida.

4.1 El problema del halo

Un filtro de convolución como el gaussiano calcula el nuevo valor de un píxel a partir de sus vecinos inmediatos. Esto genera un problema cuando la imagen se divide en regiones: los píxeles cercanos al límite entre dos regiones requieren información de píxeles que pertenecen a la región vecina. Si cada proceso recibiera únicamente su región asignada, no podría calcular correctamente el filtro para sus propias filas de borde.

Para resolver esto, cada región se extiende con un *halo*: una cantidad adicional de filas de contexto tomadas de las regiones vecinas, igual al radio del kernel utilizado (una fila arriba y una abajo para el kernel gaussiano 3×3 obligatorio, cuyo radio es 1). Estas filas de halo son datos reales de la imagen, no valores inventados, y su único propósito es brindar el contexto necesario para calcular correctamente los píxeles de la región. Una vez procesada la región, únicamente las filas correspondientes a la región propiamente asignada son devueltas; las filas de halo se descartan, evitando así que una misma fila de la imagen sea contada más de una vez en la reconstrucción final.

5 Manejo de los bordes de la imagen

Es necesario distinguir dos situaciones distintas que en apariencia son similares pero que se resuelven de forma diferente:

- **Halo entre regiones:** cuando una región necesita píxeles de una región *vecina* real (por ejemplo, la última fila de la región superior). Este caso se resuelve mediante el mecanismo de halo descrito anteriormente, ya que los datos requeridos existen y solo hace falta transportarlos.
- **Borde exterior de la imagen:** cuando el kernel requiere un píxel que está fuera de los límites de la imagen completa (por ejemplo, la fila anterior a la primera fila de la imagen). En este caso no existe ningún dato real que recuperar.

Para el segundo caso, el sistema utiliza la estrategia de **repetir el píxel más cercano**: cuando se solicita una fila o columna fuera de los límites de la imagen, se sustituye por la fila o columna válida más próxima al borde. Esta estrategia se aplicó de forma consistente y se dividió por responsabilidad entre los dos lenguajes según el eje involucrado:

- El borde **superior e inferior** de la imagen (filas) lo resuelve Erlang, ya que forma parte del mismo mecanismo de halo: al construir la región extendida de cada proceso, si la fila solicitada no existe dentro de los límites de la imagen, se repite la fila más cercana en su lugar.
- El borde **izquierdo y derecho** de la imagen (columnas) lo resuelve Scheme, de manera local, dentro de cada fila que recibe. Dado que la división de la imagen se realiza únicamente por filas, cada fila que llega a Scheme contiene siempre el ancho

completo de la imagen; por lo tanto, la información necesaria para resolver el borde de una columna extrema está siempre disponible en la misma fila, sin requerir datos de ningún otro proceso.

Esta división de responsabilidades sigue un criterio simple: Erlang resuelve lo que involucra datos de otros procesos (el halo entre regiones), mientras que Scheme resuelve lo que puede calcular exclusivamente con la información que ya tiene disponible (el borde lateral de su propia fila).

6 Modelo de concurrencia

La concurrencia del sistema se implementa utilizando el modelo de procesos ligeros y paso de mensajes propio de Erlang. A diferencia de los hilos del sistema operativo, un proceso de Erlang no comparte memoria con ningún otro proceso; la única forma de comunicación entre ellos es mediante el envío explícito de mensajes.

El proceso coordinador, luego de dividir la imagen en regiones, crea un proceso independiente por cada región mediante la primitiva `spawn`. Cada uno de estos procesos ejecuta, de forma completamente aislada y concurrente respecto a los demás, la lógica encargada de comunicarse con una instancia de Scheme para procesar su región correspondiente. Una vez obtenido el resultado, cada proceso lo envía de regreso al proceso coordinador mediante un mensaje etiquetado con la fila inicial de su región.

Debido a que los procesos se ejecutan de manera concurrente, no existe garantía sobre el orden en que sus resultados llegan al coordinador. Por esta razón, cada mensaje de resultado incluye la posición inicial de la región a la que corresponde, permitiendo que, una vez recibidos todos los resultados, estos puedan ordenarse correctamente antes de reconstruir la imagen final. De esta manera, el sistema garantiza que el desorden inherente a la ejecución concurrente no afecte la corrección del resultado.

7 Protocolo de comunicación entre Erlang y Scheme

Dado que Erlang y Scheme son entornos de ejecución independientes, la comunicación entre ambos no puede realizarse mediante una llamada directa a función, sino que requiere ejecutar Scheme como un proceso externo del sistema operativo e intercambiar información con él. Para esta comunicación se optó por un protocolo basado en archivos de texto plano, por su simplicidad de implementación y facilidad de depuración frente a alternativas como el paso de datos serializados por entrada y salida estándar.

Por cada región a procesar, Erlang genera un archivo de entrada que contiene, en su primera línea, el radio del kernel utilizado, y en las líneas siguientes, una fila de la región (incluyendo el halo) por línea, con los valores de cada píxel separados por espacios. El kernel de convolución se transmite en un archivo aparte, común a todas las regiones, con un formato equivalente. Ninguno de los dos archivos declara explícitamente la cantidad de filas que contiene: cada uno se lee completo hasta el final del archivo, lo cual es posible precisamente porque cada bloque de información reside en su propio archivo independiente.

Una vez generados los archivos de entrada, Erlang ejecuta el programa de Scheme como un proceso del sistema operativo, indicándole como argumentos las rutas del archivo de región, el archivo del kernel, y un tercer archivo donde deberá escribir su resultado. El

resultado se transmite con el mismo formato de línea por fila, conteniendo únicamente las filas correspondientes a la región asignada, sin el halo recibido.

7.1 Detección de fallos

Para detectar si la ejecución de Scheme finalizó correctamente, Erlang no depende del contenido del archivo de salida, sino del código de salida (*exit status*) del proceso del sistema operativo. Por convención, un código de salida igual a cero indica una ejecución exitosa, mientras que cualquier otro valor indica que ocurrió un error durante el procesamiento de la región. Erlang obtiene este código directamente del sistema operativo al finalizar el proceso externo, sin necesidad de que Scheme reporte el error por ningún otro medio.

Cuando se detecta que una región falló, el sistema no continúa como si nada hubiera ocurrido: informa el error indicando la región afectada y detiene la ejecución, evitando así que una falla en el procesamiento de una región produzca, de forma silenciosa, una imagen de salida incorrecta o incompleta.

8 Implementación del filtro

8.1 Filtro gaussiano (obligatorio)

El filtro gaussiano se implementó como una convolución 3×3 sobre cada canal de color por separado. El kernel utilizado es:

$$G = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Para cada píxel de la región asignada, se calcula una combinación ponderada de sus 8 vecinos y de sí mismo: cada valor se multiplica por el peso correspondiente del kernel según su posición relativa, se suman los 9 productos, y el resultado se divide entre la suma de los pesos (16). Este cálculo se realiza de forma independiente para cada uno de los tres canales (rojo, verde y azul).

La implementación en Scheme separa el proceso en pasos independientes: primero se extrae cada canal de color de la matriz de píxeles; después, cada fila se extiende repitiendo su primer y último elemento, de modo que la ventana de convolución pueda calcular también los píxeles de las columnas extremas; luego se desliza una ventana de tamaño igual al kernel sobre cada posición válida, calculando en cada una la combinación ponderada descrita; y finalmente los tres canales procesados se recombinan en una única matriz de píxeles.

8.2 Inversión de colores (transformación adicional)

Como segunda transformación, obligatoria según el enunciado, se implementó la inversión de colores. A diferencia del filtro gaussiano, esta transformación no depende de los píxeles vecinos: el nuevo valor de cada componente de color se calcula únicamente a partir de su propio valor, mediante la fórmula:

$$R' = 255 - R, \quad G' = 255 - G, \quad B' = 255 - B$$

Al no necesitar información de píxeles vecinos, esta transformación no requiere ningún mecanismo de ventana deslizante ni de manejo de bordes: simplemente se descarta el halo recibido (que en este caso no aporta información útil) y se aplica la fórmula anterior a cada píxel de la región propiamente asignada.

9 Limitaciones del sistema

- El sistema garantiza que nunca se creen más regiones que filas tiene la imagen; sin embargo, cuando la cantidad de procesos solicitada supera la altura de la imagen, el paralelismo efectivamente alcanzado es menor al solicitado, ya que la cantidad de regiones se limita a la altura disponible.
- Ante el fallo de una región, el sistema actual detiene la ejecución e informa el error, pero no reintenta automáticamente el procesamiento de la región fallida. La reejecución automática se identifica como una extensión posible del sistema, mencionada en el enunciado del proyecto.
- El intercambio de información entre Erlang y Scheme mediante archivos temporales introduce una sobrecarga adicional (escritura y lectura de disco por cada región procesada) que debe considerarse al analizar el *speedup* obtenido, dado que esta operación no se paraleliza de forma perfecta con el aumento de procesos.

10 Resultados experimentales

Para evaluar el rendimiento del sistema se midió el tiempo de ejecución utilizando dos imágenes reales de tamaño considerable y los dos filtros implementados, con 1, 2, 4 y 8 procesos Erlang en cada caso. Las imágenes utilizadas fueron:

- Imagen 1: 1080×815 píxeles (880 200 píxeles en total).
- Imagen 2: 860×792 píxeles (681 120 píxeles en total).

Todas las mediciones corresponden al tiempo real transcurrido (*wall-clock time*) del programa completo, desde la lectura del archivo de entrada hasta la escritura del archivo de salida. En las 16 combinaciones de imagen, filtro y cantidad de procesos se verificó que la imagen resultante contuviera exactamente la misma cantidad de píxeles que la imagen original, confirmando que la corrección del sistema se mantiene independientemente de la cantidad de procesos utilizada.

10.1 Filtro gaussiano

	Imagen 1 (1080×815)			Imagen 2 (860×792)		
p	$T_p(s)$	S_p	E_p	$T_p(s)$	S_p	E_p
1	19.998	1.000	1.000	14.651	1.000	1.000
2	9.867	2.027	1.013	7.419	1.975	0.987
4	6.377	3.136	0.784	5.151	2.844	0.711
8	5.494	3.640	0.455	4.789	3.059	0.382

Tabla 1: Filtro gaussiano: tiempo, speedup y eficiencia para ambas imágenes.

10.2 Filtro de inversión de colores

	Imagen 1 (1080×815)			Imagen 2 (860×792)		
p	$T_p(\text{s})$	S_p	E_p	$T_p(\text{s})$	S_p	E_p
1	8.081	1.000	1.000	6.444	1.000	1.000
2	5.948	1.359	0.679	4.861	1.326	0.663
4	5.237	1.543	0.386	3.901	1.652	0.413
8	4.679	1.727	0.216	3.920	1.644	0.205

Tabla 2: Filtro de inversión de colores: tiempo, speedup y eficiencia para ambas imágenes.

11 Análisis del speedup y la eficiencia

11.1 Comportamiento general

En ambas imágenes y ambos filtros, el speedup aumenta con la cantidad de procesos pero de forma cada vez menos pronunciada, alejándose del speedup ideal ($S_p = p$) a medida que p crece. La eficiencia, en consecuencia, decrece de forma sostenida: nunca se recupera el rendimiento perfecto de $p = 1$ al repartir el trabajo entre más procesos, un resultado esperable en cualquier sistema paralelo real.

Entre $p = 1$ y $p = 2$ se observa un speedup cercano al ideal en el filtro gaussiano (hasta levemente superior a 2 en un caso), lo cual se explica en parte por el efecto de caché de disco: al ejecutar primero la prueba con $p = 1$, el sistema operativo retiene en memoria el contenido del archivo ya leído, favoreciendo levemente las corridas posteriores. Es un recordatorio de que medir tiempos en un sistema real siempre involucra variabilidad externa al algoritmo en sí.

11.2 Gaussiano vs. inversión: el costo relativo de la sobrecarga

El hallazgo más revelador es que el filtro de **inversión de colores obtiene un speedup notablemente peor que el gaussiano**, a pesar de ser una operación mucho más simple computacionalmente. Esto se explica por la naturaleza de la sobrecarga del sistema: cada región, sin importar el filtro aplicado, requiere escribir un archivo de entrada, lanzar una instancia completamente nueva del intérprete de Racket, y leer un archivo de resultado. Este costo es aproximadamente **constante por región**, independientemente de cuánto cálculo real se realice dentro de ella.

Para el filtro gaussiano, el cálculo de la convolución 3×3 para cientos de miles de píxeles representa una porción considerable del tiempo total, por lo que repartir ese cálculo entre más procesos sí produce una mejora notable. Para la inversión de colores, en cambio, el cálculo en sí (una resta por canal, sin vecinos) es casi instantáneo comparado con el costo fijo de arrancar Racket y mover los datos por archivos — por lo que agregar más procesos reparte un trabajo que ya era pequeño, mientras la sobrecarga fija se mantiene igual o incluso aumenta (al haber más regiones, hay más instancias de Racket que arrancar en total). Este es un caso concreto y medido de un principio general de la computación paralela: **la paralelización beneficia más a las tareas donde el cálculo domina sobre la sobrecarga de coordinación**, y beneficia poco (o incluso puede perjudicar) a tareas donde ocurre lo contrario.

11.3 Un caso de rendimiento decreciente

En el filtro de inversión sobre la Imagen 2, el tiempo con $p = 8$ (3,920s) fue ligeramente **mayor** que con $p = 4$ (3,901s) — es decir, agregar más procesos empeoró el rendimiento. Esto confirma que, para tareas con poco cálculo real, existe un punto a partir del cual el costo de crear más procesos (y más instancias de Racket) supera cualquier beneficio de paralelizar un trabajo cada vez más pequeño por región.

11.4 Factores que explican el speedup por debajo del ideal

- **Arranque del intérprete de Racket:** cada región implica lanzar una instancia nueva de Scheme desde cero, con el costo de inicialización que eso conlleva, repetido una vez por región.
- **Escritura y lectura de archivos temporales:** la comunicación entre Erlang y Scheme se realiza mediante disco, no en memoria, lo cual introduce latencia de E/S por cada región.
- **Serialización de los datos a texto:** los píxeles se convierten a texto plano y se vuelven a interpretar como números en cada intercambio, un costo que no se paraleliza.
- **Costo fijo por región:** al aumentar la cantidad de procesos, cada región es más pequeña, pero el costo de arrancar Racket y mover archivos no se reduce proporcionalmente — por lo que dicho costo fijo representa una fracción cada vez mayor del tiempo de cada proceso individual.

Estos resultados evidencian que, si bien el sistema logra un paralelismo real y medible (particularmente en el filtro gaussiano, donde el cálculo domina sobre la coordinación), el diseño basado en archivos de texto y procesos externos de Scheme introduce una sobrecarga que limita el speedup alcanzable, especialmente para transformaciones computacionalmente livianas.